SPARSH-VAANI

OCR → Excel / CSV / TXT → Raspberry Pi ↔ ESP32

Code and Setup Guide

For the Multi-Sensory Medicine Label Reader and optional six-compartment smart pill dispenser

What this guide contains

1. Raspberry Pi setup and library checks
2. OCR code using OpenCV and Tesseract
3. TXT-to-CSV and Excel conversion
4. USB serial communication code for Raspberry Pi
5. ESP32 Arduino serial receiver example
6. Test steps, filenames, and important safety notes

This guide continues from your earlier OCR work. It does not require rebuilding OCR from scratch if your existing setup already works.

1. Project folder and setup

```
mkdir -p ~/Sparsh-Vaani
cd ~/Sparsh-Vaani
ls -lah
```

Put the prescription image in this folder and name it prescription.jpg, or update IMAGE_PATH in the OCR code. Check your existing OCR setup first:

```
python3 -c "import cv2, pytesseract; print('OCR imports OK')"
tesseract --version
```

Install only missing integration packages (run inside your project virtual environment if you already have one):

```
sudo apt update
sudo apt install -y tesseract-ocr tesseract-ocr-eng python3-opencv python3-numpy python3-pil python3-venv python3-pip espeak-r
python3 -m pip install pytesseract pyserial openpyxl
```

If your OS blocks system-wide pip installs, use a virtual environment with system packages enabled:

```
cd ~/Sparsh-Vaani
python3 -m venv --system-site-packages .venv
source .venv/bin/activate
python -m pip install pytesseract pyserial openpyxl
```

2. OCR code — ocr_medicine.py

Create the file with `nano ocr_medicine.py` and paste this code:

```
import cv2
import pytesseract
from pathlib import Path

IMAGE_PATH = "prescription.jpg"
OUTPUT_TXT = "medicine_ocr.txt"

image = cv2.imread(IMAGE_PATH)
if image is None:
    raise FileNotFoundError(
        f"Cannot open {IMAGE_PATH}. Check the image path."
    )

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray = cv2.resize(
    gray, None, fx=2, fy=2,
    interpolation=cv2.INTER_CUBIC
)
processed = cv2.threshold(
    gray, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)[1]

text = pytesseract.image_to_string(
    processed, config="--psm 6"
)
Path(OUTPUT_TXT).write_text(text, encoding="utf-8")

print("OCR completed. Extracted text:")
print(text)
print(f"Saved to: {OUTPUT_TXT}")
```

Run OCR:

```
python ocr_medicine.py
```

3. Convert TXT to CSV and Excel — convert_to_excel.py

This creates an OCR-results spreadsheet from non-empty text lines. It does not guarantee that every line is a verified medicine name.

Create the file with `nano convert_to_excel.py`:

```
import csv
from pathlib import Path
from openpyxl import Workbook

TXT_FILE = "medicine_ocr.txt"
CSV_FILE = "5_tablet_medicine_database.csv"
EXCEL_FILE = "5_tablet_medicine_database.xlsx"

text_path = Path(TXT_FILE)
```

```
if not text_path.exists():
    raise FileNotFoundError(
        f"{TXT_FILE} not found. Run OCR first."
    )

lines = [
    line.strip()
    for line in text_path.read_text(
        encoding="utf-8"
    ).splitlines()
    if line.strip()
]

with open(CSV_FILE, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["OCR_Extracted_Text"])
    for line in lines:
        writer.writerow([line])

workbook = Workbook()
sheet = workbook.active
sheet.title = "OCR Results"
sheet.append(["OCR_Extracted_Text"])
for line in lines:
    sheet.append([line])

sheet.column_dimensions["A"].width = 65
workbook.save(EXCEL_FILE)

print("Created:", CSV_FILE)
print("Created:", EXCEL_FILE)
print("Lines extracted:", len(lines))
```

Run conversion:

```
python convert_to_excel.py
```

4. Raspberry Pi ↔ ESP32 over USB serial

Create `nano pi_esp32.py`. Set PORT to the actual device shown on your Raspberry Pi.

```
import serial
import time

PORT = "/dev/ttyUSB0"  # May instead be /dev/ttyACM0
BAUD = 115200

ser = serial.Serial(PORT, BAUD, timeout=2)
time.sleep(2)

def send_command(command):
    ser.write((command + "\\n").encode("utf-8"))
    print("Pi -> ESP32:", command)
    response = ser.readline().decode(
        "utf-8", errors="replace"
    ).strip()
    print("ESP32 -> Pi:", response or "No response")

try:
    print("Commands: scan, dispense 1-6, quit")
    while True:
        choice = input("> ").strip().lower()

        if choice == "quit":
            break
        elif choice == "scan":
            send_command("SCAN")
        elif choice.startswith("dispense "):
            parts = choice.split()
            if len(parts) == 2 and parts[1] in "123456":
                send_command("DISPENSE," + parts[1])
            else:
                print("Enter a compartment from 1 to 6.")
        else:
            print("Use: scan, dispense 1-6, or quit")
finally:
    ser.close()
```

Find the port, then run the script:

```
ls /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
python pi_esp32.py
```

5. ESP32 Arduino serial receiver

Upload this to the ESP32 from Arduino IDE. It acknowledges commands; servo motion is intentionally not implemented until your real servo wiring and angles are confirmed.

```
String command = "";

void setup() {
  Serial.begin(115200);
  Serial.println("ESP32_READY");
}

void dispenseCompartment(int slot) {
  // Add calibrated carousel/release servo control here.
  // This acknowledgement does not prove a tablet was released.
  Serial.print("DISPENSE_REQUEST_RECEIVED,");
  Serial.println(slot);
}

void loop() {
  if (Serial.available()) {
    command = Serial.readStringUntil('\\n');
    command.trim();

    if (command == "SCAN") {
      Serial.println("SUCCESS");
    }
    else if (command.startsWith("DISPENSE,")) {
      int slot = command.substring(9).toInt();
      if (slot >= 1 && slot <= 6) {
        dispenseCompartment(slot);
      } else {
        Serial.println("ERROR_INVALID_SLOT");
      }
    }
```

```
    else {
      Serial.println("ERROR_UNKNOWN_COMMAND");
    }
  }
}
```

6. Recommended test sequence

1. Run OCR and inspect medicine_ocr.txt.
2. Run conversion and inspect the CSV/Excel rows.
3. Upload the ESP32 sketch; close Arduino Serial Monitor before using the same USB port from Python.
4. Run pi_esp32.py and enter `scan`.
5. Test `dispense 1` through `dispense 6` only after adding and checking safe servo-control code.

7. Output files

Filename	Purpose
prescription.jpg	Input image (example filename)
ocr_medicine.py	OCR and save extracted text
medicine_ocr.txt	Raw OCR text
convert_to_excel.py	Convert text lines into CSV and Excel
5_tablet_medicine_database.csv	CSV of OCR text lines
5_tablet_medicine_database.xlsx	Excel OCR-results workbook
pi_esp32.py	USB serial commands from Raspberry Pi

Safety and accuracy notes

• OCR output is not a medically verified medicine list. Confirm medicine name, strength, and instructions against the original packaging or prescription before use.
• A sensor detecting a tablet in the chute does not prove the patient swallowed it. Use a separate patient confirmation process.
• This ESP32 example only acknowledges a dispensing request; it does not drive servos or verify a tablet was released.
• Power servos from an appropriate regulated external supply; connect grounds together. Never power servos from the ESP32 3.3V pin.
• Change the serial PORT and image filename to match your actual Raspberry Pi setup.